

# Optimization for 3D Scene Reconstruction

A Comparative Study of Optimizers, Losses, and Representations  
for NeRF and 3D Gaussian Splatting

TUTORIAL #1 - PROJECT PROPOSAL

## Table of Contents

|                                             |   |
|---------------------------------------------|---|
| 1. Problem Identification .....             | 2 |
| 2. Problem Formulation .....                | 3 |
| 3. Optimization Methods .....               | 4 |
| 3.1 Scene Representations .....             | 4 |
| 3.2 Optimizers (applied to NeRF) .....      | 4 |
| 4. Analysis Plan .....                      | 5 |
| 5. Planned Improvements .....               | 5 |
| 6. Work Distribution and Deliverables ..... | 6 |

## 1. Problem Identification

3D scene reconstruction from a small set of 2D photographs is a foundational task in computer vision and AI. Given a collection of images captured from different viewpoints, the goal is to recover a representation of the underlying scene that allows new, previously unseen views to be rendered with high fidelity. The task underpins applications across robotics, augmented and virtual reality, autonomous driving, virtual production, and digital heritage preservation.

Two recent approaches have defined the state of the art:

- **Neural Radiance Fields (NeRF)**, introduced in 2020, represents the scene as a continuous function parameterised by a small multi-layer perceptron that maps a 3D point and a viewing direction to a colour and a density.
- **3D Gaussian Splatting**, introduced in 2023, represents the scene as a large collection of explicit 3D Gaussian primitives, each with a position, scale, rotation, opacity, and colour.

Both approaches frame scene reconstruction as a continuous, non-convex optimization problem: find the parameters of the chosen representation such that, when rendered from the known camera viewpoints, they best reproduce the captured photographs. This project formulates and studies that optimization problem in depth, with the goal of not only producing visually convincing 3D reconstructions, but also understanding how the choice of optimizer, loss function, regularization, and representation affects convergence behavior, training stability, and final reconstruction quality.

This problem combines several characteristics handled poorly by classical methods. It is non-convex, because the rendering operator composes the differentiable volume-rendering equation with the nonlinear MLP (NeRF) or with the non-linear Gaussian rasterization pipeline (Gaussian Splatting); even with a convex per-pixel loss, the composed objective in  $\theta$  is non-convex with many local minima and saddle points. It is high-dimensional, with  $|\theta|$  on the order of  $10^5$  parameters for a compact NeRF MLP and  $10^6$  to  $10^7$  for Gaussian Splatting on a typical scene. It is stochastic, because at each iteration only a small subset of rays (around  $10^3$  to  $10^4$  out of millions per scene) can be sampled; the per-step gradient is therefore a noisy estimate of the true gradient. And it is ill-conditioned in pixel space: textured regions of the scene produce large gradients while texture-less regions produce nearly none, so any optimizer that does not adapt or sample carefully will either over-fit easy regions or stagnate on hard ones.

The problem is also still an active area of research: the original NeRF paper triggered a still-growing body of work on accelerated variants (Instant-NGP, Plenoxels), alternative representations (3D Gaussian Splatting, triplane methods), extensions to dynamic scenes (4D Gaussian Splatting, 2024-2025, which augments each Gaussian primitive with a temporal evolution model), and improved optimization recipes (curriculum sampling, adaptive density control, learning-rate restarts).

## 2. Problem Formulation

Let  $\theta$  denote the parameters of the chosen scene representation:

- For NeRF,  $\theta$  is the set of weights and biases of a small MLP.
- For 3D Gaussian Splatting,  $\theta$  is the collection of 3D Gaussian positions, scales, rotations, opacities, and colours.

Let  $R(\theta; \pi)$  denote the differentiable rendering operator (given the parameters  $\theta$  and a camera pose  $\pi$ , it produces a synthesised image). Let  $\{(I_i, \pi_i)\}_{i=1..N}$  be the captured photographs and their associated camera poses (recovered by Structure-from-Motion).

The reconstruction problem is then:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(R(\theta; \pi_i), I_i) + \lambda \Omega(\theta)$$

where  $L$  is a per-image loss that measures the discrepancy between the rendered and observed images, and  $\Omega(\theta)$  is a regularisation term on the parameters with weight  $\lambda \geq 0$  (typically the squared L2 norm of the network weights for the NeRF MLP, or a density / opacity regulariser for Gaussian Splatting; specific candidates are listed below). Because the rendering operator is differentiable with respect to  $\theta$ , the problem admits gradient-based optimization, although the loss landscape is non-convex and high-dimensional.

The formulation choices we will make and defend explicitly:

- **The image loss  $L$ .** Candidates include L1, L2, SSIM, and convex combinations of these. Each has known effects on which image details are preserved and how robust the optimization is to outliers.
- **The regularisation term  $\Omega(\theta)$ .** Candidates include weight decay (for the NeRF MLP), and density/opacity regularisers and adaptive density control heuristics (for Gaussian Splatting).
- **The view- and ray-sampling strategy.** Which rays or which views are presented to the optimizer at each iteration. Uniform random, importance sampling by current reconstruction error, and curriculum schedules are options.
- **The parameterisation choices.** Positional encoding frequency for NeRF; initial Gaussian distribution and density-control thresholds for Gaussian Splatting.

For each ray cast through a pixel, the rendering operator samples  $N$  positions along the ray between near and far planes, queries the scene representation at each position to obtain a density  $\sigma_k \geq 0$  and an RGB colour  $c_k \in [0, 1]^3$ , and composites them into a per-pixel colour using the discretised volume-rendering equation:

$$C(\mathbf{r}) = \sum_{k=1}^N T_k (1 - \exp(-\sigma_k \delta_k)) c_k, \quad T_k = \exp\left(-\sum_{j < k} \sigma_j \delta_j\right)$$

where  $\delta_k = t_{k+1} - t_k$  is the distance between consecutive samples and  $T_k$  is the accumulated transmittance up to sample  $k$ . The operator is differentiable with respect to every  $\sigma_k$  and  $c_k$ , and through the representation also with respect to  $\theta$ . The full objective is therefore amenable to gradient-based optimization via reverse-mode automatic differentiation (PyTorch autograd composes the gradient through the loss, the renderer, the encoding, and the MLP in one backward pass).

Given the properties of the problem, first-order stochastic methods are more suited than second-order ones, as they would be infeasible due to scale and size or not appropriated due to high gradient noise.

To allow high frequency scene detail, the input coordinate is wrapped in a fixed (non-learned) positional encoding:

$$\gamma(x) = [x, \sin(2^0 \pi x), \cos(2^0 \pi x), \dots, \sin(2^{(L-1)} \pi x), \cos(2^{(L-1)} \pi x)]$$

with  $L$  between 8 and 14 frequency bands, as implicit bias in small fully connected networks is toward low-frequency functions of their inputs. Although the encoding adds no parameters to be optimized, by changing the input to the MLP high-frequency variations in the scene become easier for the optimizer to handle, with the  $L$  parameter affecting reconstruction quality, as lesser values unable fine detail and greater values cause overfitting and instability.

### 3. Optimization Methods

We will implement and compare methods along two axes: the choice of scene representation, and the choice of optimizer.

#### 3.1 Scene Representations

**NeRF, implemented from scratch.** We will write a compact implementation of the NeRF representation and the volume rendering integration in PyTorch. The implementation will be small (on the order of a few hundred to a few thousand lines of clear Python), so that every optimization choice remains under our control.

**3D Gaussian Splatting, using a published reference implementation.** We will use an open-source Gaussian Splatting implementation as a contemporary point of comparison.

#### 3.2 Optimizers (applied to NeRF)

We will implement five optimizers ourselves on top of PyTorch's automatic differentiation, plus a learning-rate schedule, and compare them on the NeRF reconstruction problem:

- Stochastic Gradient Descent (SGD)
- SGD with classical momentum
- SGD with Nesterov accelerated gradient
- Adam
- AdamW

For each method we give the per-iteration update rule, what aspect of the optimization problem the method is designed to address, and what its inclusion contributes to the comparison.

**Learning-rate schedule.** On top of the best-performing optimizer, we will apply a cosine-annealing schedule with a short warmup. Schedules of this form have become standard in deep learning because they explore the loss surface aggressively early in training (large  $\eta$ ) and refine near a minimum late in training (small  $\eta$ ). Their effect on this non-convex stochastic problem is one of the things the comparison will quantify.

By developing custom implementations of the optimizers instead of pre-existing ones, it is possible to have greater control and understanding, ensuring a greater depth of analysis when observing the optimization process and ability to compare in equal terms each optimizer.

## 4. Analysis Plan

For each combination of representation, optimizer, and loss, we will measure and report:

### Convergence behaviour:

- Training loss as a function of iteration count
- Rendered-image PSNR and SSIM on held-out views as a function of iteration count
- Wall-clock time to reach a fixed target quality

### Stability:

- Sensitivity to random initialization across multiple seeds
- Sensitivity to learning rate (sweeps)
- Behavior on scenes of different difficulty (textured vs textureless, dense vs sparse view coverage)

### Final quality:

- PSNR and SSIM on held-out test views
- Qualitative side-by-side comparison of rendered novel views

All numerical comparisons will be averaged across at least three seeds, and we will report variability alongside means.

## 5. Planned Improvements

Based on the comparison results from Section 4, we will attempt to investigate the four improvements below. Each is motivated by a specific property of the optimization problem identified in Section 2.

**Adaptive view sampling.** Standard NeRF training samples rays uniformly across the training images. This wastes computation on already-well-reconstructed regions and underservices regions with high residual error.

**Multi-scale (coarse-to-fine) training.** A direct way to mitigate the non-convexity of the loss is to begin optimization on a smoother, lower-resolution version of the problem and progressively refine it.

**Learning-rate restarts (warm restarts, SGDR).** To escape sharp local minima in the non-convex landscape, we will replace the single cosine-annealing schedule of Section 3.2 with a cyclic cosine schedule.

**Perceptual loss.** Pixel-wise losses (L1, L2) are poorly aligned with human perception of image quality: an image that is uniformly slightly off in colour can have the same MSE as one with a sharp edge displaced by a pixel, but the two look very different.

## 6. Work Distribution and Deliverables

The workload is split equally across the project's several areas, with joint participation in code review, tutorials, and synchronization meetings. The final deliverable is a Jupyter Notebook accompanied by a written report derived from it.